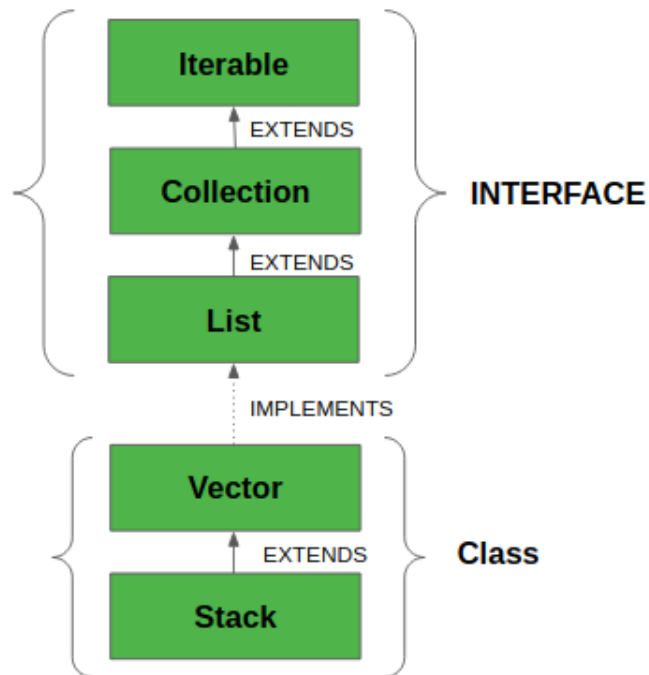


Java Collection Framework

Stack Class in Java :-

- The Stack class is in the java.util package.
- It represents a LIFO (Last-In-First-Out) data structure.
- It extends the Vector class.
- Allows duplicates and maintains insertion order.



Syntax :-

```
Stack<Type> stack = new Stack<Type>();
```

Methods of Stack :-

Method	Description
push(E item)	Adds an element to the top of the stack
pop()	Removes and returns the top element
peek()	Returns the top element without removing it
isEmpty()	Checks whether the stack is empty
search(Object o)	Returns the 1-based position from the top of the stack

Example :-

```
package collections;

import java.util.Iterator;
import java.util.Stack;

public class stack_demo {

    public static void main(String[] args) throws InterruptedException {
        //using Non-Generic Stack
        Stack s=new Stack();

        s.push(101); //used to add object in stck
        s.push("Sagar");
        s.push(99.9);
        s.push('A');

        System.out.println("Non-Generic stack Elements: " +s);
        s.pop();
        System.out.println("After pop stack is: "+s);

        s.remove(1);
        System.out.println("After remove stack is: "+s);

        System.out.println("capacity of stack: "+s.capacity());
        System.out.println("After peak stack is: "+s.peek());
        System.out.println("After search: "+s.search(99.9));

        System.out.println("Iterating Stack Elements:");
        Iterator it=s.iterator();
        for(Object obj:s)
        {
            System.out.println(obj);
            Thread.sleep(1000);
        }

        //using Generic Stack
        Stack<Integer>s1=new Stack<Integer>();

        s1.push(12);
        s1.push(24);
        s1.push(36);
        s1.push(48);

        System.out.println("Generic Stack Elements: " +s1);

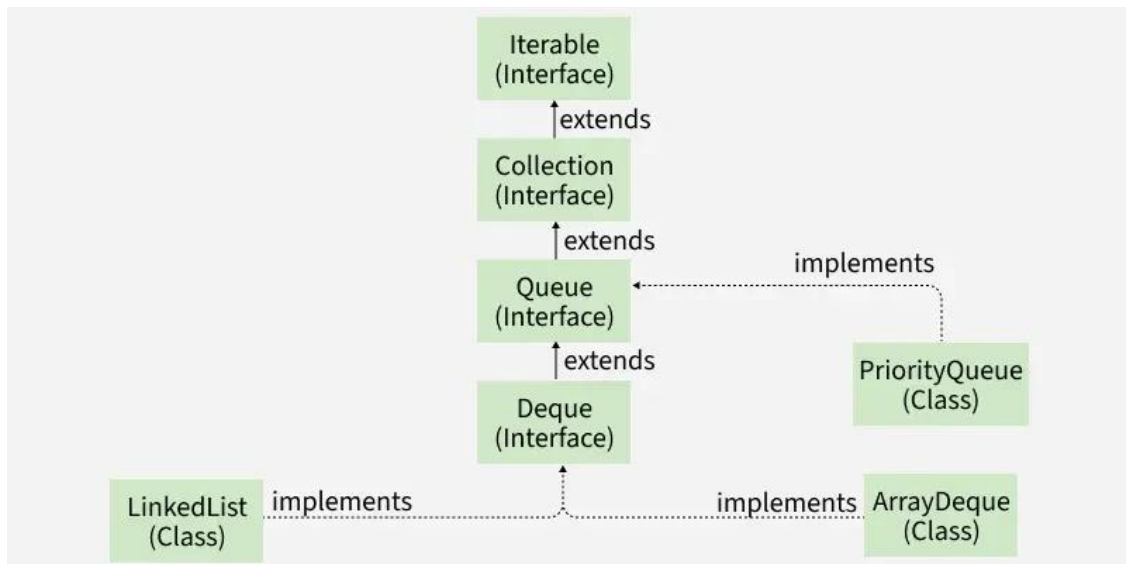
        System.out.println("Iterating Stack Elements:");
        Iterator it1=s1.iterator();
        while(it1.hasNext())
        {
            System.out.println(it1.next());
            Thread.sleep(1000);
        }
    }
}
```

Output :-

```
Non-Generic stack Elements: [101, Sagar, 99.9, A]
After pop stack is: [101, Sagar, 99.9]
After remove stack is: [101, 99.9]
capacity of stack: 10
After peak stack is: 99.9
After search: 1
Iterating Stack Elements:
101
99.9
Generic Stack Elements: [12, 24, 36, 48]
Iterating Stack Elements:
12
24
36
48
```

Queue Interface In Java :-

- The Queue interface is part of the java.util package.
- It is used to store elements in a FIFO (First-In-First-Out) order.
- The element that is inserted first is removed first.
- It extends the Collection interface.
- It is used when elements need to be processed in order.



Declaration of Queue Interface :-

```
Queue<Obj> queue = new LinkedList<Obj>();
```

Common Implementations of Queue Interface:

1. **LinkedList**: FIFO queue, allows nulls, implements Queue & Deque.
2. **ArrayDeque**: Fast, resizable array queue, no nulls allowed.
3. **PriorityQueue**: Orders elements by priority, not insertion.
4. **ConcurrentLinkedQueue**: Thread-safe, non-blocking queue.
5. **BlockingQueue**: Thread-safe, supports blocking for producer-consumer.

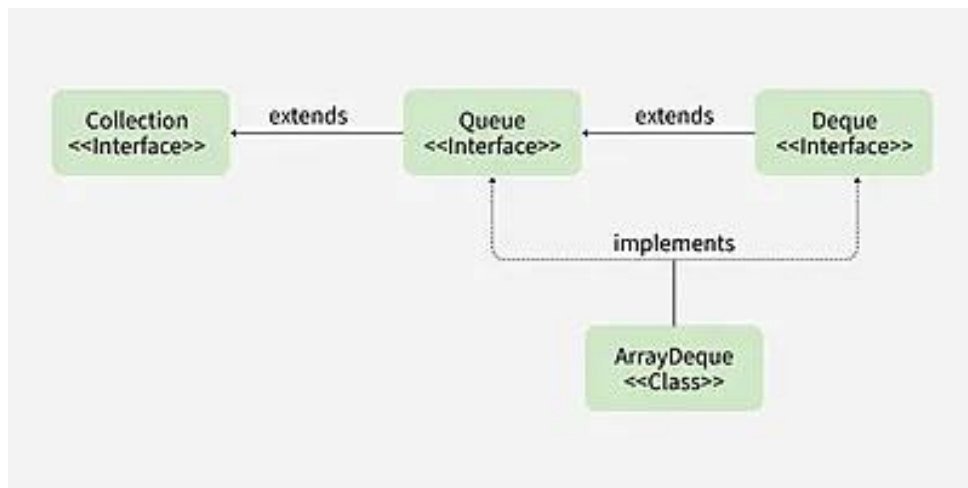
Methods of Queue Interface :-

Method	Description (Short)
add(E e)	Adds element; throws exception if fails.
offer(E e)	Adds element; returns false if fails.
remove()	Removes head; throws exception if empty.
poll()	Removes head; returns null if empty.

peek()	Returns head; null if empty.
size()	Returns number of elements.
isEmpty()	Checks if queue is empty.
contains(Object o)	Checks if element exists.
iterator()	Returns iterator of queue.
toArray()	Converts queue to array.
addFirst(E e)	Adds at front (Deque only).
addLast(E e)	Adds at end (Deque only).
offerFirst(E e)	Adds at front; false if fails (Deque).
offerLast(E e)	Adds at end; false if fails (Deque).
removeFirst()	Removes first element (Deque).
removeLast()	Removes last element (Deque).
pollFirst()	Removes first; null if empty (Deque).
pollLast()	Removes last; null if empty (Deque).
getFirst()	Gets first; doesn't remove (Deque).
getLast()	Gets last; doesn't remove (Deque).
peekFirst()	Peeks first; null if empty (Deque).
peekLast()	Peeks last; null if empty (Deque).
put(E e)	Inserts; waits if full (BlockingQueue).
take()	Removes head; waits if empty (BlockingQueue).

1. ArrayDeque :-

- ArrayDeque is a resizable array-based deque.
- It allows insertion and deletion from both ends.
- It does not allow null elements.
- It is faster than LinkedList.



Syntax :-

ArrayDeque<Type> deque = new ArrayDeque<Type>();

Methods of ArrayDeque :-

Method	Description
add(E e)	Adds element at the end.
addFirst(E e)	Adds element at the front.
addLast(E e)	Adds element at the end.
offer(E e)	Inserts element; returns false if fails.
offerFirst(E e)	Inserts element at front; returns false if fails.
offerLast(E e)	Inserts element at end; returns false if fails.
remove()	Removes and returns head element.
removeFirst()	Removes and returns first element.
removeLast()	Removes and returns last element.
poll()	Removes head; returns null if empty.
pollFirst()	Removes first; returns null if empty.
pollLast()	Removes last; returns null if empty.
peek()	Retrieves head; returns null if empty.
peekFirst()	Retrieves first; returns null if empty.
peekLast()	Retrieves last; returns null if empty.
getFirst()	Returns first element; throws exception if empty.
getLast()	Returns last element; throws exception if empty.
clear()	Removes all elements.
size()	Returns number of elements.

Example :-

```
package collections;

import java.util.ArrayDeque;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class queue_demo {

    public static void main(String[] args) throws InterruptedException {
        //using Non-Generic Queue
        Queue q=new ArrayDeque();

        q.add(101);
        q.add("Sagar");
        q.add(99.9);
        q.add('A');

        System.out.println("Non-Generic Queue Elements: " +q);

        System.out.println("head of object: "+q.element());

        q.offer("%");
        System.out.println("After queue is: "+q);

        System.out.println("peek: "+q.peek());
        System.out.println("After peek is: "+q);

        System.out.println("poll: "+q.poll());
        System.out.println("After poll is: "+q);

        System.out.println("Iterating Queue Elements: ");
        for (Object obj:q)
        {
            System.out.println(obj);
            Thread.sleep(1000);
        }

        //using Generic Queue
        Queue<String>q1=new LinkedList<String>();

        q1.add("Sagar");
        q1.add("Rahul");
        q1.add("Prajwal");
        q1.add("Ganesh");

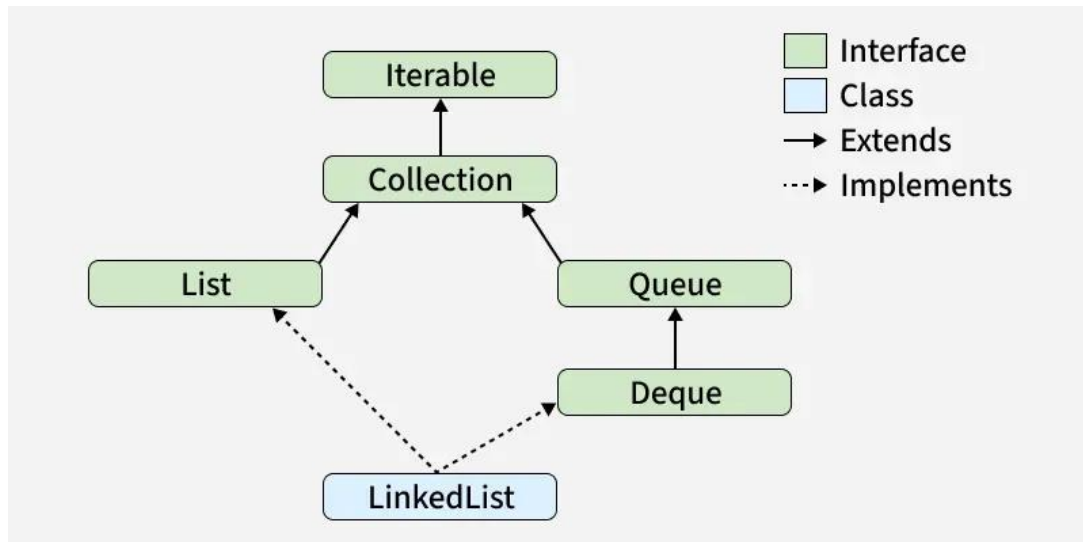
        System.out.println("Generic Queue Elements: " +q1);
    }
}
```

Output :-

```
Non-Generic Queue Elements: [101, Sagar, 99.9, A]
head of object: 101
After queue is: [101, Sagar, 99.9, A, %]
peek: 101
After peek is: [101, Sagar, 99.9, A, %]
poll: 101
After poll is: [Sagar, 99.9, A, %]
Iterating Queue Elements:
Sagar
99.9
A
%
Generic Queue Elements: [Sagar, Rahul, Prajwal, Ganesh]
```

2. LinkedList :-

- LinkedList is a doubly linked list from the java.util package.
- It implements the List and Deque interfaces.
- It allows duplicates, nulls, and insertion/deletion from both ends.
- It is not synchronized and faster than ArrayList for add/remove operations.



Syntax :-

LinkedList<Type> list = new LinkedList<Type>();

Methods of LinkedList in Deque :-

Method	Description
addFirst(E e)	Adds element at the front of the list.
addLast(E e)	Adds element at the end of the list.
offerFirst(E e)	Inserts element at the front; returns false if fails.
offerLast(E e)	Inserts element at the end; returns false if fails.
removeFirst()	Removes and returns the first element.
removeLast()	Removes and returns the last element.
pollFirst()	Removes and returns first; returns null if empty.
pollLast()	Removes and returns last; returns null if empty.
getFirst()	Returns the first element; throws exception if empty.
getLast()	Returns the last element; throws exception if empty.
peekFirst()	Retrieves first element; returns null if empty.
peekLast()	Retrieves last element; returns null if empty.
push(E e)	Pushes an element onto the stack represented by the deque.
pop()	Pops an element from the stack represented by the deque.
descendingIterator()	Returns an iterator over elements in reverse order.

Example:-

```
package collections;

import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class queue_linkedlist_demo {

    public static void main(String[] args) throws InterruptedException {

        //using Non-Generic collection of Queue
        Queue q=new LinkedList();
        q.add(101); //add elements in LinkedList
        q.add("Sagar");
        q.add(99.9);
        q.add('A');

        System.out.println("Non-Generic collection of Queue is: "+q);

        //Head element without removing
        System.out.println("Head element (element): "+q.element());

        //Add new element
        q.offer("%");
        System.out.println("After offer(%): "+q);

        //View head without removing
        System.out.println("Peek element: "+q.peek());
        System.out.println("After peek: " +q);

        //Remove head
        System.out.println("Removed element (poll): "+q.poll());
        System.out.println("After poll: "+q);

        //Iterating queue elements
        System.out.println("Iterating Non-Generic Queue: ");
        for(Object obj:q)
        {
            System.out.println(obj);
            Thread.sleep(1000);
        }

        //using Generic collection of Queue
        Queue<String>q1=new LinkedList<String>();

        q1.add("Sagar");
        q1.add("Rahul");
        q1.add("Prajwal");
        q1.add("Ganesh");

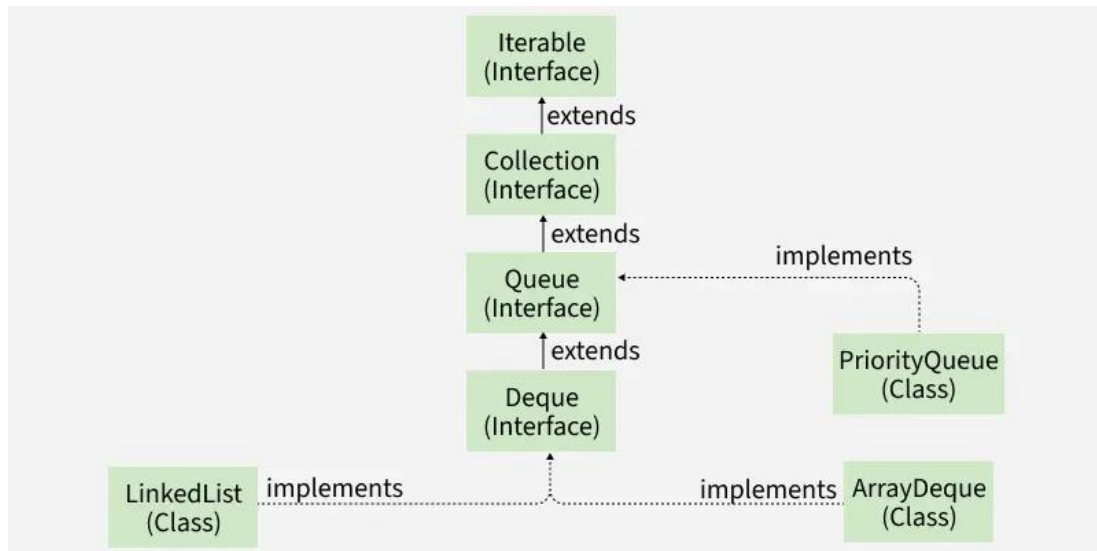
        System.out.println("Generic collecton of Queue is: "+q1);
    }
}
```

Output :-

```
Non-Generic collection of Queue is: [101, Sagar, 99.9, A]
Head element (element): 101
After offer(%): [101, Sagar, 99.9, A, %]
Peek element: 101
After peek: [101, Sagar, 99.9, A, %]
Removed element (poll): 101
After poll: [Sagar, 99.9, A, %]
Iterating Non-Generic Queue:
Sagar
99.9
A
%
Generic collection of Queue is: [Sagar, Rahul, Prajwal, Ganesh]
```

3. PriorityQueue :-

- PriorityQueue is part of the java.util package.
- It stores elements based on priority, not insertion order.
- It does not allow nulls but allows duplicates.
- It is not thread-safe and is used in job scheduling and algorithms.



Syntax :-

PriorityQueue<Type> pq = new PriorityQueue<Type>();

Methods PriorityQueue :-

Method	Description
add(E e)	Inserts element; throws exception if fails.
offer(E e)	Inserts element; returns false if fails.
peek()	Returns head element; null if empty.
poll()	Removes and returns head; null if empty.
remove()	Removes head element; throws exception if empty.
contains(Object o)	Checks if element exists.
clear()	Removes all elements.
size()	Returns number of elements.
isEmpty()	Checks if queue is empty.
iterator()	Returns iterator over elements.

Example :-

```
package collections;

import java.util.Iterator;
import java.util.PriorityQueue;
import java.util.Queue;

public class priorityqueue_demo {

    public static void main(String[] args) throws InterruptedException {
        //using Non-Generic collection of PriorityQueue
        Queue pq = new PriorityQueue();
        pq.add(45); //add elements in PriorityQueue
        pq.add(10);
        pq.add(30);
        pq.add(25);

        System.out.println("Non-Generic collection of PriorityQueue is: "+pq);

        System.out.println("Head element (peek): "+pq.peek()); //Head element(without removing)

        System.out.println("Removed element (poll): "+pq.poll()); //Removing head element
        System.out.println("After poll,PriorityQueue: "+pq);

        pq.offer(5); //Adding new element
        System.out.println("After offer(5): "+pq);

        //Iterating elements
        System.out.println("Iterating Non-Generic PriorityQueue: ");
        for(Object obj:pq)
        {
            System.out.println(obj);
            Thread.sleep(1000);
        }

        //using Generic collection of PriorityQueue
        Queue<String> pq1 = new PriorityQueue<String>();
        pq1.add("Sagar");
        pq1.add("Rahul");
        pq1.add("Prajwal");
        pq1.add("Ganesh");

        System.out.println("Generic collection of PriorityQueue is: "+pq1);
    }
}
```

Output :-

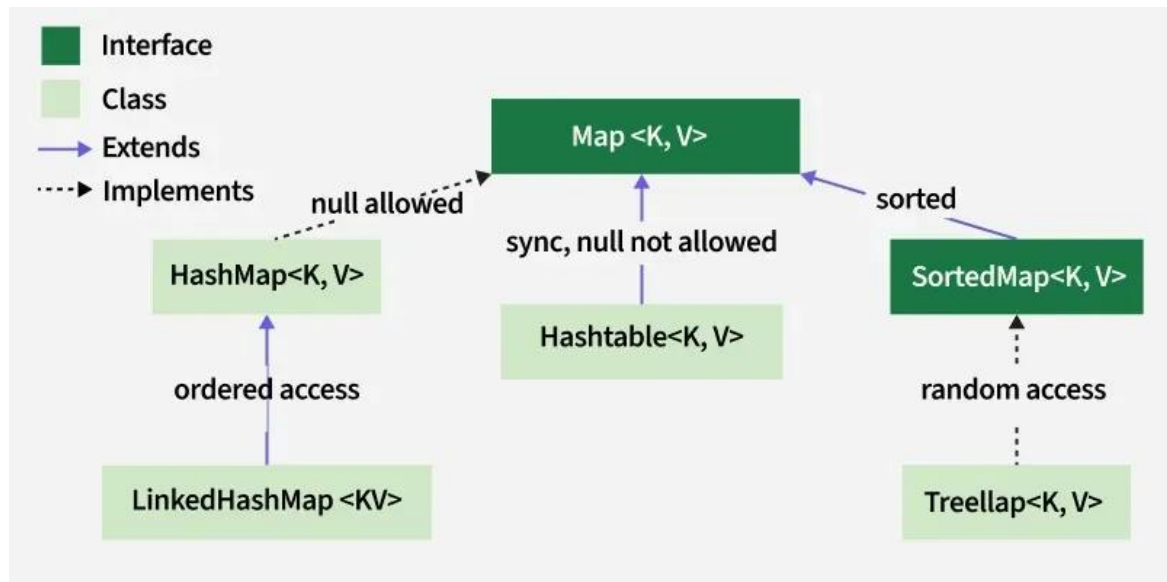
```
Non-Generic collection of PriorityQueue is: [10, 25, 30, 45]
Head element (peek): 10
Removed element (poll): 10
After poll,PriorityQueue: [25, 45, 30]
After offer(5): [5, 25, 30, 45]
Iterating Non-Generic PriorityQueue:
5
25
30
45
Generic collection of PriorityQueue is: [Ganesh, Prajwal, Rahul, Sagar]
```

Difference Between ArrayDeque, LinkedList & PriorityQueue :-

Feature	ArrayDeque	LinkedList	PriorityQueue
Package	java.util	java.util	java.util
Order of Elements	Maintains FIFO or LIFO order	Maintains insertion order	Orders elements by priority
Null Elements	Not allowed	Allowed	Not allowed
Underlying Data Structure	Resizable array	Doubly linked list	Heap structure
Performance	Fast insertion/removal at both ends	Slower than ArrayDeque	Efficient for priority-based access
Use as Stack/Queue	Can act as both stack and queue	Can act as list, queue, or deque	Used as a priority queue only
Thread-Safety	Not synchronized	Not synchronized	Not synchronized

Map Interface in Java :-

- The Map interface is part of the java.util package.
- It stores data in key-value pairs, where keys are unique.
- It does not extend the Collection interface.
- Common implementations are HashMap, LinkedHashMap, TreeMap, and Hashtable.



Declaration of Map Interface :-

```
Map<String, Integer> hm = new HashMap<>();
```

Common Implementations of Map Interface :-

1. **HashMap:** Stores key-value pairs using hashing; fast access and insertion.
2. **LinkedHashMap:** Maintains insertion order of keys.
3. **TreeMap:** Stores entries in sorted order of keys.
4. **Hashtable:** Synchronized map; no null keys or values allowed.

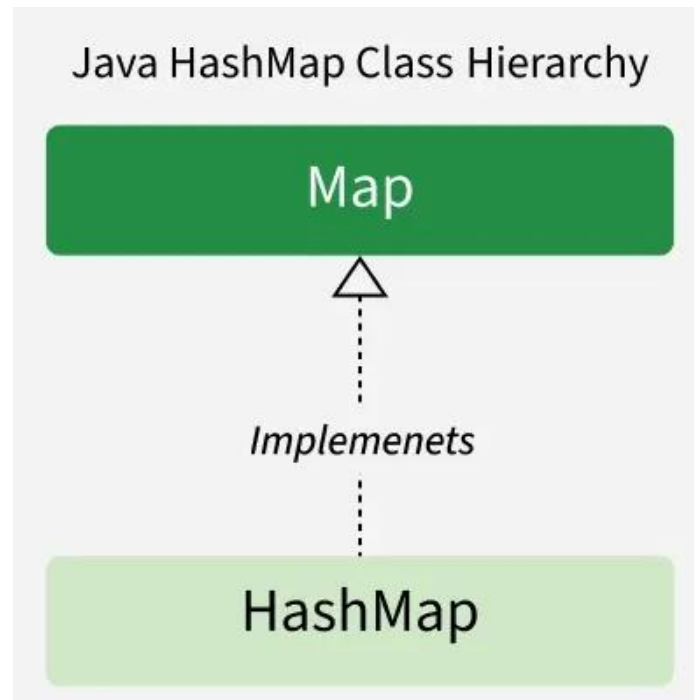
Methods of Map Interface :-

Method	Description
put(K key, V value)	Adds or updates a key-value pair.
get(Object key)	Returns value for the specified key.
remove(Object key)	Removes the entry for the specified key.
containsKey(Object key)	Checks if key exists in map.
containsValue(Object value)	Checks if value exists in map.
keySet()	Returns a Set view of all keys.
values()	Returns a Collection view of all values.
entrySet()	Returns a Set of key-value pairs.

isEmpty()	Checks if map is empty.
size()	Returns number of key-value pairs.
clear()	Removes all mappings from map.

1. HashMap :-

- The HashMap class is part of the java.util package and implements the Map interface.
- It stores data in key–value pairs using hashing for fast access.
- Duplicate keys are not allowed, but duplicate values are permitted.
- It does not maintain insertion order and is not synchronized.



Syntax :-

```
HashMap<KeyType, ValueType> map = new HashMap<KeyType, ValueType>();
```

Methods of HashMap :-

Method	Description
put(K key, V value)	Adds or updates a key-value pair.
get(Object key)	Returns value for the specified key.
remove(Object key)	Removes mapping for the specified key.
containsKey(Object key)	Checks if key exists in map.
containsValue(Object value)	Checks if value exists in map.
keySet()	Returns a set of all keys.
values()	Returns a collection of all values.
entrySet()	Returns a set of key-value mappings.
size()	Returns number of mappings.
isEmpty()	Checks if map is empty.
clear()	Removes all key-value pairs.

Example :-

```
package collections;

import java.util.HashMap;

public class hashmap_demo {

    public static void main(String[] args) {
        //using Non-Genric Collection of HashMap
        HashMap hsm=new HashMap();
        hsm.put(101,"Sagar"); //add elements in HashMap
        hsm.put(102,'A');
        hsm.put(103,12345678901);
        hsm.put(104,77.89);

        System.out.println("Non-Generic collecton of HashMap is: "+hsm);

        System.out.println("contains value: "+hsm.containsValue('A'));
        System.out.println("contains key: "+hsm.containsKey(103));
        System.out.println("keyset: "+hsm.keySet());

        System.out.println("remove key: "+hsm.remove(101));
        System.out.println("remove key value: "+hsm.remove(102,'A'));
        System.out.println("After removing key value: "+hsm);

        System.out.println("replace key: "+hsm.replace(103,98765432101));
        System.out.println("After replacing key: "+hsm);

        System.out.println("replace old value: "+hsm.replace(104,77.89,85.11));
        System.out.println("After replacing old value: "+hsm);

        //using Non-Genric Collection of HashMap
        HashMap<Integer,String>emp_data=new HashMap<Integer,String>();
        emp_data.put(1,"Sagar");
        emp_data.put(2,"Omkar");
        emp_data.put(3,"Sahil");

        emp_data.putIfAbsent(2,"Shubham");

        System.out.println("Non-Generic collecton of HashMap is:"+emp_data);
    }
}
```

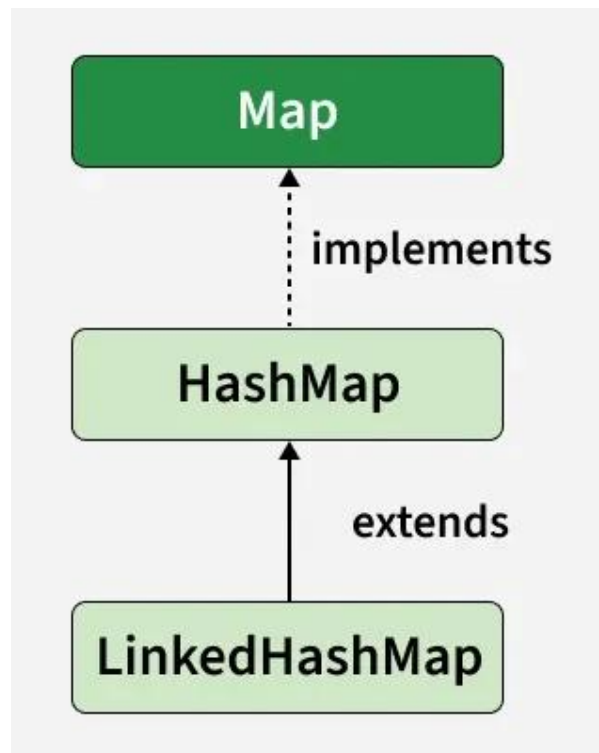
Output :-

```
Non-Generic collecton of HashMap is: {101=Sagar, 102=A, 103=1234567890, 104=77.89}
contains value: true
contains key: true
keyset: [101, 102, 103, 104]
remove key: Sagar
```

```
remove key value: true
After removing key value: {103=1234567890, 104=77.89}
replace key: 1234567890
After replacing key: {103=9876543210, 104=77.89}
replace old value: true
After replacing old value: {103=9876543210, 104=85.11}
Non-Generic collection of HashMap is:{1=Sagar, 2=Omkar, 3=Sahil}
```

2. LinkedHashMap :-

- The LinkedHashMap class is part of the java.util package.
- It stores data in key–value pairs like HashMap but maintains insertion order.
- Duplicate keys are not allowed, but duplicate values are permitted.
- It allows one null key, multiple null values, and is not synchronized.



Syntax :-

```
LinkedHashMap<KeyType, ValueType> map = new LinkedHashMap<KeyType, ValueType>();
```

Methods of LinkedHashMap :-

Method	Description
put(K key, V value)	Inserts or updates a key–value pair.
get(Object key)	Returns the value for the specified key.
remove(Object key)	Removes the mapping for the given key.
containsKey(Object key)	Checks if the map contains the specified key.
containsValue(Object value)	Checks if the map contains the specified value.
keySet()	Returns a set view of all keys.
values()	Returns a collection view of all values.
entrySet()	Returns a set of key–value mappings.
size()	Returns the number of key–value pairs.
isEmpty()	Checks if the map is empty.
clear()	Removes all mappings from the map.

getOrDefault(Object key, V defaultValue)	Returns the value for key or a default if key not found.
replace(K key, V value)	Replaces the value for a specified key.
putIfAbsent(K key, V value)	Inserts value only if key is not already mapped.

Example :-

```
package collections;

import java.util.LinkedHashMap;
import java.util.Set;

public class linkedhashmap_demo {

    public static void main(String[] args) {
        //using Non-Generic LinkedHashMap
        LinkedHashMap lhm=new LinkedHashMap();
        lhm.put(1, "Pune"); // Adding elements in LinkedHashMap
        lhm.put(2, "Mumbai");
        lhm.put(3, "Nagpur");
        lhm.put(4, "Delhi");

        System.out.println("Non-Generic collection of LinkedHashMap is: "+lhm);
        System.out.println("Value at key 2: "+lhm.get(2)); //Accessing value

        lhm.remove(3); //Removing an entry
        System.out.println("After removing key 3: "+lhm);

        lhm.replace(4,"Delhi","Hyderabad"); //Replacing value
        System.out.println("After replacing key 4 value: "+lhm);

        System.out.println("Contains key: "+lhm.containsKey(1));
        System.out.println("Contains value: "+lhm.containsValue("Pune"));

        // Iterating keys and values
        Set keys=lhm.keySet();
        System.out.println("Iterating LinkedHashMap Elements: ");
        for (Object k:keys)
        {
            System.out.println(k+"="+lhm.get(k));
        }

        //using Generic LinkedHashMap
        LinkedHashMap<Integer,String>lhm1=new LinkedHashMap<Integer,String>();

        lhm1.put(101, "SBI");
        lhm1.put(102, "HDFC");
        lhm1.put(103, "ICICI");
        lhm1.put(104, "AXIS");

        System.out.println("Generic collection of LinkedHashMap is: "+lhm1);
    }
}
```

Output :-

```
Non-Generic collection of LinkedHashMap is: {1=Pune, 2=Mumbai, 3=Nagpur, 4=Delhi}
Value at key 2: Mumbai
After removing key 3: {1=Pune, 2=Mumbai, 4=Delhi}
After replacing key 4 value: {1=Pune, 2=Mumbai, 4=Hyderabad}
Contains key: true
Contains value: true
Iterating LinkedHashMap Elements:
1=Pune
2=Mumbai
4=Hyderabad
Generic collection of LinkedHashMap is: {101=SBI, 102=HDFC, 103=ICICI, 104=AXIS}
```

3. TreeMap :-

- The TreeMap class is part of the java.util package and implements the NavigableMap interface.
- It stores data in key–value pairs in sorted (ascending) order of keys.
- Duplicate keys are not allowed, but duplicate values are permitted.
- It does not allow null keys and is not synchronized.



Syntax :-

```
TreeMap<KeyType, ValueType> map = new TreeMap<KeyType, ValueType>();
```

Methods of TreeMap :-

Method	Description
put(K key, V value)	Inserts or updates a key–value pair.
get(Object key)	Returns the value for the specified key.
remove(Object key)	Removes the mapping for the given key.
containsKey(Object key)	Checks if the map contains the specified key.
containsValue(Object value)	Checks if the map contains the specified value.
firstKey()	Returns the first (lowest) key.
lastKey()	Returns the last (highest) key.
headMap(K key)	Returns a view of the map with keys less than the given key.
tailMap(K key)	Returns a view of the map with keys greater than or equal to the given key.
subMap(K fromKey, K toKey)	Returns a view of the map between two keys.
keySet()	Returns a set of all keys.
values()	Returns a collection of all values.

entrySet()	Returns a set of key–value mappings.
size()	Returns the number of key–value pairs.
clear()	Removes all mappings from the map.

Example :-

```
package collections;

import java.util.Set;
import java.util.TreeMap;

public class treemap_demo {

    public static void main(String[] args) {
        //using Non-Generic TreeMap
        TreeMap tm=new TreeMap();
        tm.put(1,"Sagar"); //add elements in treemap
        tm.put(2,"Omkar");
        tm.put(3,"Sahil");

        System.out.println("Non-Generic collecton of TreeMap is:"+tm);

        System.out.println("Value at key 2: " +tm.get(2)); //Accessing value

        tm.remove(3); // Removing element
        System.out.println("After removing key 3: " +tm);

        tm.replace(1,"Rahul"); // Replacing value
        System.out.println("After replacing key 1 value: " +tm);

        Set keys=tm.keySet();
        System.out.println("\nIterating TreeMap:");
        for(Object k:keys)
        {
            System.out.println(k+"="+tm.get(k));
        }

        //using Non-Generic TreeMap
        TreeMap<Integer,String>tm1=new TreeMap<Integer,String>();

        tm1.put(101, "SBI");
        tm1.put(102, "HDFC");
        tm1.put(103, "ICICI");
        tm1.put(104, "AXIS");

        System.out.println("Generic collection of TreeMap is: "+tm1);

    }
}
```

Output :-

Non-Generic collection of TreeMap is: {1=Sagar, 2=Omkar, 3=Sahil}

Value at key 2: Omkar

After removing key 3: {1=Sagar, 2=Omkar}

After replacing key 1 value: {1=Rahul, 2=Omkar}

Iterating TreeMap:

1=Rahul

2=Omkar

Generic collection of TreeMap is: {101=SBI, 102=HDFC, 103=ICICI, 104=AXIS}

Difference Between HashMap, LinkedHashMap & TreeMap :-

Feature	HashMap	LinkedHashMap	TreeMap
Order of Elements	No order maintained	Maintains insertion order	Maintains sorted order of keys
Underlying Data Structure	Hash table	Hash table + Linked list	Red-Black tree
Null Keys / Values	Allows one null key and multiple null values	Allows one null key and multiple null values	Does not allow null keys, allows null values
Performance	Fastest for insertion, deletion, lookup	Slightly slower than HashMap	Slower due to sorting
Sorting	Not sorted	Not sorted	Sorted by keys (natural/custom order)
Thread-Safety	Not synchronized	Not synchronized	Not synchronized
Use Case	Best for quick access	Best for maintaining insertion order	Best for sorted key storage